Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
— Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

«СЛУЖБА АВТОМАТИЧЕСКОГО РЕЗЕРВНОГО КОПИРОВАНИЯ ДЛЯ MS WINDOWS»

по дисциплине «БСИТ»

Выполнил студент <подпись> Волошкевич М.А.
гр.5151001/40202

Преподаватель / ассистент <подпись> Дахнович А.Д.

Санкт-Петербург 2026г.

1. Цель работы.

Получить навыки разработки служб операционной системы Microsoft Windows. Изучить принципы взаимодействия приложения с диспетчером служб Windows (Service Control Manager, SCM), освоить регистрацию, запуск и остановку службы, а также обработку управляющих команд.

Разработать службу автоматического резервного копирования заданного каталога. Предусмотреть настройку параметров работы через конфигурационный файл, фильтрацию файлов по маскам имён, создание ZIP-архива без использования сторонних архиваторов и автоматическое обновление резервной копии при изменении содержимого исходного каталога. Обеспечить возможность работы с локальными и сетевыми путями.

2. Архитектура

2.1. Выбор архитектуры системы

Разрабатываемая система предназначена для автоматического резервного копирования содержимого заданного каталога в ZIP-архив. Программа реализована в виде службы Windows, работающей в фоновом режиме и не требующей постоянного взаимодействия с пользователем.

Основная задача службы заключается в наблюдении за исходным каталогом, обнаружении изменений файловой системы и своевременном обновлении резервной копии. Параметры работы службы задаются во внешнем конфигурационном файле, что позволяет изменять исходный каталог, расположение архива и набор обрабатываемых файлов без перекомпиляции программы.

В качестве основного механизма управления используется Windows Service Control Manager. Служба регистрируется в системе, получает команды запуска и остановки, а также сообщает диспетчеру о текущем состоянии.

При проектировании были приняты следующие архитектурные решения, приведённые в Таблица 1.

Вопрос	**Решение**	**Обоснование**
Форма приложения	Служба Windows	Служба предназначена для постоянной фоновой работы и должна запускаться и останавливаться средствами операционной системы. В отличие от обычного консольного приложения, она управляется через SCM и не требует открытого пользовательского сеанса.
Хранение настроек	Внешний файл backup.ini	Путь к исходному каталогу, путь к архиву и маски файлов можно изменять независимо от исходного кода. Это упрощает настройку программы на различных компьютерах.

Обнаружение изменений	ReadDirectoryChangesW	Функция Windows позволяет получать уведомления об изменениях в каталоге без постоянного полного сканирования всех файлов. Это снижает объём лишних операций при обычной работе.
Обработка пропущенных событий	Периодическое сканирование	Дополнительная проверка каталога позволяет обнаружить изменения, уведомления о которых могли быть пропущены.
Формирование архива	Самостоятельная реализация ZIP	Архив создаётся непосредственно средствами программы. Использование внешних архиваторов и дополнительных программных зависимостей не требуется.
Обновление резервной копии	Полное пересоздание ZIP	При обновлении архив формируется заново на основании текущего состояния исходного каталога. Это позволяет учитывать создание, изменение и удаление файлов.
Управление службой	Windows SCM	SCM отвечает за регистрацию службы, запуск, остановку и передачу управляющих команд. Дополнительно реализованы команды командной строки для управления службой.

Таблица 1. Основные архитектурные решения системы

2.2. Общая структура программы

Программа состоит из нескольких логических компонентов, взаимодействующих между собой.

Компонент	**Назначение**
`Config`	Структура хранения параметров: исходный каталог, путь к архиву, маски файлов, задержка обработки изменений и интервал повторного сканирования.

`parseConfig`	Чтение и разбор конфигурационного файла backup.ini. Проверка наличия обязательных параметров и установка минимальных значений временных интервалов.
`wildcardMatch`	Сопоставление имени файла с маской. Поддерживаются символы * и ?, сравнение выполняется без учёта регистра.
`createZip`	Рекурсивный обход исходного каталога, отбор подходящих файлов, создание временного ZIP-архива и его последующая замена.
`addFileToZip`	Чтение содержимого отдельного файла, вычисление CRC32 и запись локального заголовка и данных файла в архив.
`watchWorker`	Фоновый поток наблюдения за исходным каталогом с использованием ReadDirectoryChangesW. При обнаружении изменений инициирует обновление архива.
`runWorker`	Создание события остановки и рабочего потока, а также выполнение периодической проверки исходного каталога.
`serviceMain`	Точка входа службы, регистрация обработчика управляющих команд и передача SCM информации о состоянии службы.
`controlHandlerEx`	Обработка команд остановки и завершения работы, поступающих от SCM.
`manageService`	Установка, удаление, запуск, остановка, перезапуск и получение состояния службы через функции Windows API.
`logLine`	Запись диагностических сообщений в файл WinBackupService.log.

Таблица 2. Основные компоненты программной реализации

2.3. Конфигурация системы

Настройка параметров выполняется через файл backup.ini, расположенный в одном каталоге с исполняемым файлом службы. Путь к конфигурационному файлу формируется автоматически на основе расположения исполняемого файла.

Для настройки используются следующие параметры:

Параметр	**Тип значения**	**Назначение**
`source_dir`	Путь	Абсолютный путь к каталогу, содержимое которого необходимо резервировать.
`archive_path`	Путь	Полный путь к создаваемому ZIP-архиву.
`patterns`	Список масок	Перечень масок имён файлов, разделённых точкой с запятой. В архив включаются только соответствующие файлы.
`debounce_ms`	Целое число	Задержка в миллисекундах перед обновлением архива после обнаружения изменения. Используется для объединения нескольких близких по времени событий.
`rescan_seconds`	Целое число	Интервал периодической повторной проверки исходного каталога в секундах.

Таблица 3. Параметры конфигурационного файла

Параметры `source_dir`, `archive_path` и `patterns` являются обязательными. Если хотя бы один из них отсутствует, программа не может начать работу. Для временных параметров предусмотрены значения по умолчанию: задержка обработки изменений составляет 1500 мс, а период повторного сканирования — 60 секунд. При этом минимально допустимая задержка

ограничена 100 мс, а минимальный интервал повторного сканирования составляет 5 секунд.

В качестве примера конфигурации можно использовать следующие значения:

- `source_dir` — `C:\Backup\Documents`;
- `archive_path` — `C:\Backup\archive\backup.zip`;
- `patterns` — `*.txt;*.docx;*.pdf;*.jpg;*.png`;
- `debounce_ms` — `1500`;
- `rescan_seconds` — `10`.

Файл конфигурации не требует отдельного секционного заголовка. Каждая настройка задаётся отдельной строкой в формате «имя параметра = значение». Строки, начинающиеся с символов `#` или `;`, игнорируются.

2.4. Сопоставление файлов с масками

Для определения того, должен ли файл попасть в резервную копию, реализована функция сопоставления с шаблоном. Она поддерживает два специальных символа:

- `*` — соответствует последовательности символов произвольной длины, в том числе пустой;
- `?` — соответствует ровно одному произвольному символу.

Сопоставление выполняется без учёта регистра. Например, маска `*.txt` соответствует файлам `report.txt`, `notes.TXT` и `document.txt`.

Каждая маска проверяется относительно имени файла и его относительного пути внутри исходного каталога. Это позволяет использовать как маски расширений, так и шаблоны, содержащие относительные пути.

При обходе каталога программа рассматривает только обычные файлы. Каталоги самостоятельно в ZIP не добавляются, однако содержащиеся в них подходящие файлы включаются в архив благодаря рекурсивному обходу.

2.5. Формирование ZIP-архива

Создание резервной копии реализовано самостоятельно, без обращения к сторонним архиваторам. Для записи используется формат ZIP с методом хранения STORE, то есть содержимое файлов записывается без дополнительного сжатия.

Формирование архива выполняется в несколько этапов.

1. Проверяется доступность исходного каталога. Если каталог отсутствует или недоступен, создание архива прекращается с записью сообщения в журнал.
2. Создаётся каталог, в котором должен находиться архив, если он ещё не существует.
3. Создаётся временный файл с расширением `.tmp`. В него последовательно записываются данные файлов и служебные структуры ZIP.
4. Исходный каталог рекурсивно обходится с помощью `std::filesystem::recursive_directory_iterator`. Для каждого обычного файла проверяется соответствие одной из заданных масок.
5. Для каждого выбранного файла вычисляется CRC32, определяется относительный путь внутри исходного каталога и формируется локальный заголовок ZIP.
6. После записи файлов формируется центральный каталог ZIP, содержащий сведения о записях архива и смещениях их локальных заголовков.
7. В конец файла записывается запись End of Central Directory, завершающая структуру ZIP-архива.
8. Временный файл закрывается. Предыдущий архив удаляется, после чего временный файл переименовывается в целевой архив.

Для вычисления контрольной суммы реализована функция CRC32. Она обрабатывает содержимое файла побитово с использованием полинома `0xEDB88320`. Полученное значение записывается в заголовки ZIP и позволяет архиваторам проверять целостность данных.

Имена файлов внутри архива сохраняются в кодировке UTF-8. Для этого используется функция `WideCharToMultiByte`, преобразующая строки

Windows из UTF-16 в UTF-8. В заголовках ZIP устанавливается соответствующий флаг кодировки.

При добавлении файла программа сохраняет его относительный путь, поэтому структура подкаталогов воспроизводится внутри архива. Например, файл `Documents\Reports\report.txt` будет находиться в архиве по пути `Reports/report.txt`.

Полное пересоздание архива позволяет поддерживать его содержимое в соответствии с текущим состоянием исходного каталога. Изменённые файлы записываются с актуальным содержимым, новые файлы добавляются, а удалённые файлы больше не включаются в новую резервную копию.

2.6. Наблюдение за изменениями файловой системы

Для автоматического обнаружения изменений используется функция Windows API `ReadDirectoryChangesW`. Она позволяет получать уведомления о событиях файловой системы для открытого каталога.

Исходный каталог открывается с помощью `CreateFileW` с флагом `FILE_FLAG_BACKUP_SEMANTICS`, необходимым для работы с каталогами, и `FILE_FLAG_OVERLAPPED`, обеспечивающим асинхронное выполнение операции.

Для получения уведомлений используется следующий набор типов изменений:

- `FILE_NOTIFY_CHANGE_FILE_NAME` — изменение имён файлов;
- `FILE_NOTIFY_CHANGE_DIR_NAME` — изменение имён каталогов;
- `FILE_NOTIFY_CHANGE_SIZE` — изменение размера файла;
- `FILE_NOTIFY_CHANGE_LAST_WRITE` — изменение времени последней записи;
- `FILE_NOTIFY_CHANGE_CREATION` — создание файлов и каталогов.

Параметр рекурсивного наблюдения включён, поэтому изменения отслеживаются не только в корневом каталоге, но и во вложенных каталогах.

Рабочий поток ожидает наступления одного из двух событий: уведомления об изменении файловой системы или сигнала остановки службы. Для асинхронной операции используется структура `OVERLAPPED` и отдельный объект события.

После получения уведомления программа ожидает заданный интервал `debounce_ms`. Такая задержка позволяет объединить несколько событий, возникающих при одной операции записи файла, и не пересоздавать архив после каждого отдельного изменения.

После завершения задержки вызывается функция `createZip`, которая формирует новую резервную копию.

2.7. Периодическая проверка каталога

Одних уведомлений файловой системы недостаточно для обеспечения устойчивой работы: при определённых условиях события могут быть потеряны или не обработаны. Поэтому в программе предусмотрен дополнительный механизм периодической проверки.

Основной поток функции `runWorker` ожидает сигнал остановки с интервалом в одну секунду и учитывает прошедшее время. Когда значение достигает `rescan_seconds`, запускается полное пересоздание архива.

Таким образом, система использует два механизма обновления:

- событийный — для быстрого реагирования на изменения;
- периодический — для восстановления актуального состояния резервной копии при пропуске уведомлений.

При этом оба механизма используют одну функцию создания архива. Для предотвращения одновременной записи нескольких потоков применяется `std::mutex`, защищающий операцию обновления ZIP.

2.8. Работа службы Windows

Для интеграции с операционной системой используется Windows Service Control Manager. Служба имеет имя `WinBackupService` и отображаемое имя `Windows ZIP Backup Service`.

Точка входа `wmain` определяет, какую операцию необходимо выполнить. Если передан аргумент управления, вызывается функция `manageService`. В противном случае программа пытается зарегистрироваться в SCM через `StartServiceCtrlDispatcherW`.

Основные функции Windows API, используемые для взаимодействия со службой:

Функция	**Назначение**
`OpenSCManagerW`	Открытие диспетчера служб Windows.
`CreateServiceW`	Создание и регистрация новой службы в системе.
`OpenServiceW`	Получение дескриптора уже зарегистрированной службы.

StartServiceW	Запрос запуска службы.
ControlService	Передача управляющих команд, в том числе команды остановки.
DeleteService	Удаление регистрации службы.
QueryServiceStatusEx	Получение текущего состояния службы.
RegisterServiceCtrlHandlerExW	Регистрация функции обработки управляющих команд от SCM.
SetServiceStatus	Передача диспетчеру информации о текущем состоянии службы.
StartServiceCtrlDispatcherW	Подключение процесса к диспетчеру служб и запуск функции ServiceMain.

Таблица 4. Используемые функции Windows API для управления службой

При запуске через SCM вызывается функция `serviceMain`. Она регистрирует обработчик команд через `RegisterServiceCtrlHandlerExW`, сообщает состояние `SERVICE_START_PENDING`, проверяет конфигурацию и доступность исходного каталога, а затем переводит службу в состояние `SERVICE_RUNNING`.

Обработчик `controlHandlerEx` принимает команды `SERVICE_CONTROL_STOP` и `SERVICE_CONTROL_SHUTDOWN`. При получении одной из них служба сообщает о переходе в состояние `SERVICE_STOP_PENDING` и устанавливает событие остановки. Рабочие потоки получают этот сигнал и завершают выполнение.

После завершения рабочего потока служба сообщает SCM о состоянии `SERVICE_STOPPED`.

2.9. Команды управления

Для упрощения управления службой реализована обработка командной строки.

Команда	**Назначение**
install	Регистрация службы в SCM. Создаётся служба с типом `SERVICE_WIN32_OWN_PROCESS` и режимом запуска `SERVICE_DEMAND_START`.
remove	Остановка службы и удаление её регистрации.

`start`	Передача SCM команды запуска службы.
`stop`	Передача SCM команды остановки службы.
`restart`	Остановка службы, ожидание перехода в состояние `SERVICE_STOPPED` и последующий запрос запуска.
`status`	Получение и вывод текущего состояния службы.
`debug`	Запуск рабочего механизма в консольном режиме без регистрации в SCM. Используется для отладки и проверки резервного копирования.

Таблица 5. Команды управления программой

Команда `debug` предназначена для тестирования логики резервного копирования в среде разработки. Она позволяет проверить чтение конфигурации, создание ZIP и реакцию на изменения файловой системы без необходимости установки службы.

Следует учитывать, что команда `debug` не заменяет полноценный запуск через SCM: корректность регистрации, запуска и остановки службы должна проверяться отдельно в операционной системе Windows.

2.10. Журналирование

Для диагностики работы программы реализована функция `logLine`. Она записывает сообщения в файл `WinBackupService.log`, расположенный рядом с исполняемым файлом.

В журнал заносятся сведения о запуске и остановке службы, невозможности чтения конфигурации, недоступности исходного каталога, ошибках создания ZIP и результатах обновления архива.

Каждая запись снабжается локальными датой и временем. Это позволяет определить последовательность событий и установить причину возникновения ошибок при работе программы.

3. Тестирование системы

3.1. Среда тестирования

Разработка и первоначальная проверка программы выполнялись в операционной системе Linux с использованием компилятора MinGW-w64. Для запуска полученного исполняемого файла Windows использовалась среда совместимости Wine.

Программа собиралась с использованием стандарта C++17, поскольку в реализации применяется библиотека `std::filesystem`. Для упрощения запуска через Wine была использована статическая компоновка стандартных библиотек GCC.

Проверка выполнялась в консольном режиме `debug`, который запускает рабочий механизм резервного копирования без регистрации службы в SCM.

3.2. Проверка конфигурационного файла

На первом этапе был создан тестовый каталог, содержащий исходные файлы и отдельную папку для резервного архива. В конфигурационном файле были заданы следующие параметры:

- исходный каталог — `Z:\home\red\wine-backup-test\source`;
- путь к архиву — `Z:\home\red\wine-backup-test\archive\backup.zip`;
- маски файлов — `*.txt;*.docx;*.pdf;*.jpg;*.png`;
- задержка обработки изменений — 1500 мс;
- интервал повторного сканирования — 10 секунд.

После запуска программы проверялась возможность чтения конфигурации и создания резервной копии по указанному пути.

Результат: программа сформировала ZIP-архив в каталоге `archive`. В архив был включён файл `demo.txt`, соответствующий заданной маске `*.txt`.



Рис. 1. Тестовый каталог с исполняемым файлом, конфигурацией, исходными данными и каталогом архива



Рис. 2. Содержимое конфигурационного файла backup.ini

3.3. Проверка создания ZIP-архива

Для проверки создания архива в исходном каталоге был создан текстовый файл `demo.txt`, содержащий тестовую строку. После запуска программы был сформирован архив `backup.zip`.

Корректность результата проверялась средствами утилиты `unzip`. С помощью команды просмотра содержимого архива проверялось наличие файла `demo.txt`, а с помощью извлечения его содержимого — соответствие данных исходному файлу.

В результате был получен ZIP-архив, содержащий выбранный файл. Это подтверждает работоспособность механизма формирования локальных заголовков, центрального каталога и завершающей записи ZIP.

Место для рисунка

Рис. 3. Результат просмотра содержимого созданного ZIP-архива

Место для рисунка

Рис. 4. Проверка содержимого файла demo.txt внутри архива

3.4. Проверка автоматического обновления архива

На следующем этапе проверялась реакция программы на изменение исходного файла.

После первоначального создания архива в файл `demo.txt` была добавлена новая строка. Для проверки использовалась операция дописывания данных в конец файла. После обнаружения изменения и завершения интервала задержки программа сформировала обновлённый архив.

При проверке содержимого ZIP было установлено, что в нём присутствует актуальное содержимое файла, включая добавленную строку.

Таким образом, была проверена работа механизма обнаружения изменений и последующего пересоздания резервной копии.

Место для рисунка

Рис. 5. Изменение исходного файла demo.txt

Место для рисунка

Рис. 6. Обновлённое содержимое demo.txt после извлечения из ZIP

3.5. Проверка фильтрации файлов

Для проверки фильтрации в исходном каталоге были размещены файлы с различными расширениями. В конфигурации использовались маски `*.txt`, `*.docx`, `*.pdf`, `*.jpg` и `*.png`.

Файл `demo.txt` соответствовал одной из заданных масок и был включён в архив. Дополнительно был создан файл `app.log`, расширение которого отсутствовало в списке разрешённых масок.

Файл `app.log` не должен попадать в резервную копию, поскольку не соответствует ни одному из заданных шаблонов. Это позволяет ограничивать состав архива только необходимыми типами файлов.



Рис. 7. Исходный каталог с файлами разных расширений



Рис. 8. Содержимое архива после проверки фильтрации

3.6. Проверка журналирования

Для диагностики работы программы предусмотрено создание файла `WinBackupService.log` рядом с исполняемым файлом.

В журнал записываются сообщения о создании и обновлении архива, а также диагностические сведения при возникновении ошибок. Наличие журнала позволяет анализировать работу программы без необходимости отслеживать внутреннее выполнение каждого потока.



Рис. 9. Журнал WinBackupService.log после выполнения тестирования

3.7. Проверки, требующие выполнения в Windows

Использование Wine позволяет проверить основные операции с файлами, конфигурацией и формированием ZIP-архива, однако не обеспечивает полноценную проверку всех возможностей Windows Service Control Manager.

Поэтому для окончательной проверки соответствия требованиям необходимо выполнить дополнительные испытания в Windows 7–10:

- установить службу командой `install`;
- убедиться в появлении службы в оснастке `services.msc`;
- запустить службу командой `start`;
- проверить её состояние командой `status`;
- изменить файл в исходном каталоге и проверить обновление архива;
- остановить службу командой `stop`;
- проверить перезапуск командой `restart`;
- удалить регистрацию службы командой `remove`;
- проверить доступ к исходным каталогам и архивам, расположенным на сетевых ресурсах UNC.

Для проверки UNC-путей необходимо использовать сетевой путь вида `\\server\share\folder`. При этом учётная запись, от имени которой работает служба, должна обладать необходимыми правами на чтение исходного каталога и запись в каталог архива.

Перечисленные проверки являются отдельным этапом испытаний в Windows и не рассматриваются как подтверждённые результатами запуска через Wine.

4. Ограничения реализации

В процессе разработки были учтены особенности формата ZIP и используемых механизмов Windows.

Реализация использует классический формат ZIP без поддержки ZIP64. Поэтому размер отдельного файла ограничен значением 4 ГиБ, а некоторые поля структуры архива имеют ограничения классического формата. Файлы, превышающие допустимый размер, пропускаются.

Метод STORE не выполняет сжатие данных, а только упаковывает их в ZIP-контейнер. Это упрощает реализацию, но не уменьшает размер резервной копии относительно исходных файлов.

Для обновления архива сначала создаётся временный файл, после чего предыдущий архив удаляется и временный файл переименовывается в целевой. Такой способ позволяет не записывать данные непосредственно в существующий архив, однако операция замены не является полностью атомарной.

При работе службы с сетевыми каталогами необходимо учитывать права учётной записи службы. Само наличие поддержки UNC-путей со стороны файловых API не гарантирует доступ к сетевому ресурсу при запуске службы от стандартной системной учётной записи.

Также необходимо учитывать, что наблюдение за изменениями и периодическое сканирование являются двумя независимыми механизмами. Периодическая проверка служит дополнительной гарантией актуальности архива, но не заменяет обработку событий файловой системы.

5. Выводы

В ходе выполнения лабораторной работы была разработана программа автоматического резервного копирования каталогов в операционной системе Microsoft Windows. Программа реализована в виде службы, взаимодействующей с Service Control Manager.

Была реализована настройка системы через внешний конфигурационный файл, включающая выбор исходного каталога, расположения ZIP-архива, масок файлов и временных параметров обработки изменений.

Для обнаружения изменений файловой системы использована функция Windows API `ReadDirectoryChangesW`. Дополнительно реализовано периодическое сканирование исходного каталога, позволяющее восстановить актуальное состояние резервной копии при пропуске уведомлений.

Создание ZIP-архива реализовано самостоятельно, без использования сторонних архиваторов. Были реализованы вычисление контрольной суммы CRC32, запись локальных заголовков, центрального каталога и завершающей записи ZIP. При каждом обновлении архива его содержимое формируется заново на основании текущего состояния исходного каталога.

Также реализованы механизмы регистрации службы, обработки управляющих команд, изменения состояния службы и ведения журнала работы.

В среде Wine была проверена работа конфигурации, создание ZIP-архива, отбор файлов по маскам и обновление резервной копии после изменения исходного файла. Полноценная проверка взаимодействия с SCM и работы с UNC-путями требует отдельного тестирования в Windows.

В результате работы были получены навыки разработки фоновых служб Windows, использования Win32 API для управления службами и наблюдения за файловой системой, а также работы с файловыми форматами и средствами диагностики приложений.

Приложения

Приложение А. Конфигурационный файл

В приложении представлен пример конфигурации, используемой для настройки службы резервного копирования.

Параметр	Пример значения
source_dir	Z:\home\red\wine-backup-test\source
archive_path	Z:\home\red\wine-backup-test\archive\backup.zip
patterns	*.txt;*.docx;*.pdf;*.jpg;*.png
debounce_ms	1500
rescan_seconds	10

Таблица 6. Пример параметров backup.ini

Приложение Б. Команды управления

Для управления службой используются следующие команды:

- `WinBackupService.exe install` — регистрация службы;
- `WinBackupService.exe start` — запуск службы;
- `WinBackupService.exe stop` — остановка службы;
- `WinBackupService.exe restart` — перезапуск службы;
- `WinBackupService.exe status` — получение состояния;
- `WinBackupService.exe remove` — удаление службы;
- `WinBackupService.exe debug` — запуск рабочего механизма в режиме отладки.

Приложение В. Структура ZIP-архива

Создаваемый архив состоит из локальных заголовков файлов, записанных данных, центрального каталога и завершающей записи End of Central Directory.

Для каждого файла сохраняются его относительное имя, контрольная сумма CRC32, размер, дата и время, а также смещение локального заголовка относительно начала архива.

Такая структура позволяет стандартным архиваторам распознавать созданные программой ZIP-файлы и извлекать содержащиеся в них данные.

Приложение Г. Файл WinBackupService.cpp

```
#define UNICODE
#define _UNICODE
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <winsvc.h>
#include <shlwapi.h>
#include <shellapi.h>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <algorithm>
#include <cstdint>
#include <cstdio>
#include <cwctype>
#include <iostream>
#include <mutex>

#pragma comment(lib, "Advapi32.lib")
#pragma comment(lib, "Shlwapi.lib")

namespace fs = std::filesystem;
static const wchar_t* SERVICE_NAME = L"WinBackupService";
static const wchar_t* SERVICE_DISPLAY = L"Windows ZIP Backup
Service";

static SERVICE_STATUS_HANDLE g_statusHandle = nullptr;
static SERVICE_STATUS g_status{};
static HANDLE g_stopEvent = nullptr;
static std::wstring g_configPath;
static std::wstring g_logPath;
static std::mutex g_archiveMutex;

struct Config
{
```

```
  std::wstring source;
  std::wstring archive;
  std::vector<std::wstring> patterns;
  DWORD debounceMs = 1500;
  DWORD rescanSeconds = 60;
};

static std::wstring trim(std::wstring s)
{
  const wchar_t* ws = L" \t\r\n";
  size_t a = s.find_first_not_of(ws);
  if (a == std::wstring::npos)
    return L"";
  size_t b = s.find_last_not_of(ws);
  return s.substr(a, b - a + 1);
}
static std::wstring lower(std::wstring s)
{
  std::transform(s.begin(), s.end(), s.begin(),
           [](wchar_t c) { return (wchar_t)towlower(c); });
  return s;
}
static std::wstring exeDir()
{
  std::wstring p(32768, L'\0');
  DWORD n = GetModuleFileNameW(nullptr, p.data(),
(DWORD)p.size());
  p.resize(n);
  fs::path x(p);
  return x.parent_path().wstring();
}
static std::wstring joinPath(const std::wstring& a, const
std::wstring& b)
{
  return (fs::path(a) / fs::path(b)).wstring();
}
static void logLine(const std::wstring& msg)
{
  if (g_logPath.empty())
    g_logPath = joinPath(exeDir(), L"WinBackupService.log");
```

```
  SYSTEMTIME t;
  GetLocalTime(&t);
  std::wofstream f(fs::path(g_logPath), std::ios::app);
  if (f)
    f << L"[" << t.wYear << L"-" << t.wMonth << L"-" << t.wDay <<
L" "
      << t.wHour << L":" << t.wMinute << L":" << t.wSecond << L"]
" << msg
      << L"\n";
}
static bool parseConfig(const std::wstring& path, Config& c)
{
  std::wifstream f{fs::path(path)};
  if (!f)
  {
    logLine(L"Cannot open configuration: " + path);
    return false;
  }
  std::wstring line;
  while (std::getline(f, line))
  {
    line = trim(line);
    if (line.empty() || line[0] == L'#' || line[0] == L';')
      continue;
    size_t eq = line.find(L'=');
    if (eq == std::wstring::npos)
      continue;
    std::wstring key = lower(trim(line.substr(0, eq)));
    std::wstring value = trim(line.substr(eq + 1));
    if (key == L"source_dir")
      c.source = value;
    else if (key == L"archive_path")
      c.archive = value;
    else if (key == L"patterns")
    {
      std::wstringstream ss(value);
      std::wstring item;
      while (std::getline(ss, item, L';'))
        if (!(item = trim(item)).empty())
          c.patterns.push_back(item);
```

```
    }
    else if (key == L"debounce_ms")
    {
      try
      {
        c.debounceMs = (DWORD)std::stoul(value);
      }
      catch (...)
      {
      }
    }
    else if (key == L"rescan_seconds")
    {
      try
      {
        c.rescanSeconds = (DWORD)std::stoul(value);
      }
      catch (...)
      {
      }
    }
  }
  if (c.source.empty() || c.archive.empty() || c.patterns.empty())
  {
    logLine(L"Config requires source_dir, archive_path and
patterns.");
    return false;
  }
  if (c.debounceMs < 100)
    c.debounceMs = 100;
  if (c.rescanSeconds < 5)
    c.rescanSeconds = 5;
  return true;
}
static bool wildcardMatch(const wchar_t* pattern, const wchar_t*
text)
{
  // '*' matches any sequence; '?' matches one character; case-
insensitive.
  while (*pattern)
```

```
  {
    if (*pattern == L'*')
    {
      while (*pattern == L'*')
        ++pattern;
      if (!*pattern)
        return true;
      for (const wchar_t* p = text;; ++p)
      {
        if (wildcardMatch(pattern, p))
          return true;
        if (!*p)
          break;
      }
      return false;
    }
    if (!*text)
      return false;
    if (*pattern != L'?' && towlower(*pattern) != towlower(*text))
      return false;
    ++pattern;
    ++text;
  }
  return *text == L'\0';
}
static bool matches(const Config& c, const fs::path& p)
{
  const std::wstring name = p.filename().wstring();
  const std::wstring rel = p.filename().wstring();
  for (const auto& pat : c.patterns)
  {
    if (wildcardMatch(pat.c_str(), name.c_str()) ||
      wildcardMatch(pat.c_str(), rel.c_str()))
      return true;
    // A pattern containing a relative subpath is matched against
the
    // source-relative path below.
  }
  return false;
}
```

```
static uint32_t crc32(const unsigned char* data, size_t n)
{
  uint32_t c = 0xFFFFFFFFu;
  for (size_t i = 0; i < n; ++i)
  {
    c ^= data[i];
    for (int k = 0; k < 8; ++k)
      c = (c >> 1) ^ (0xEDB88320u & (0u - (c & 1u)));
  }
  return ~c;
}
static void put16(std::ofstream& f, uint16_t v)
{
  char b[2] = {(char)(v & 255), (char)((v >> 8) & 255)};
  f.write(b, 2);
}
static void put32(std::ofstream& f, uint32_t v)
{
  char b[4] = {(char)(v & 255), (char)((v >> 8) & 255),
         (char)((v >> 16) & 255), (char)((v >> 24) & 255)};
  f.write(b, 4);
}
static std::string utf8(const std::wstring& w)
{
  if (w.empty())
    return {};
  int n = WideCharToMultiByte(CP_UTF8, 0, w.c_str(),
(int)w.size(), nullptr,
                0, nullptr, nullptr);
  std::string s(n, '\0');
  WideCharToMultiByte(CP_UTF8, 0, w.c_str(), (int)w.size(),
s.data(), n,
            nullptr, nullptr);
  return s;
}
struct ZipEntry
{
  std::string name;
  uint32_t crc, size, offset;
  uint16_t time, date;
```

```
};
static void dosTimeDate(const fs::file_time_type& ft, uint16_t&
tm,
            uint16_t& dt)
{
  (void)ft;
  SYSTEMTIME st;
  GetLocalTime(&st);
  tm = (uint16_t)(((st.wHour & 31) << 11) | ((st.wMinute & 63) <<
5) |
          ((st.wSecond / 2) & 31));
  dt = (uint16_t)((((st.wYear - 1980) & 127) << 9) | ((st.wMonth &
15) << 5) |
          (st.wDay & 31));
}
static bool addFileToZip(std::ofstream& out, const fs::path& file,
             const fs::path& source, std::vector<ZipEntry>&
entries)
{
  std::ifstream in(file, std::ios::binary);
  if (!in)
  {
    logLine(L"Cannot read file: " + file.wstring());
    return false;
  }
  std::vector<unsigned char>
data((std::istreambuf_iterator<char>(in)),
                  std::istreambuf_iterator<char>());
  if (data.size() > 0xFFFFFFFFull)
  {
    logLine(L"Skipping file larger than 4 GiB: " +
file.wstring());
    return true;
  }
  std::error_code ec;
  fs::path rel = fs::relative(file, source, ec);
  if (ec)
    rel = file.filename();
  std::wstring relw = rel.wstring();
  std::replace(relw.begin(), relw.end(), L'\\', L'/');
```

```
  std::string name = utf8(relw);
  if (name.size() > 65535)
    return false;
  ZipEntry e{};
  e.name = name;
  e.crc = crc32(data.data(), data.size());
  e.size = (uint32_t)data.size();
  e.offset = (uint32_t)out.tellp();
  dosTimeDate(fs::last_write_time(file, ec), e.time, e.date);
  put32(out, 0x04034b50);
  put16(out, 20);
  put16(out, 0x0800);
  put16(out, 0); // UTF-8, STORE
  put16(out, e.time);
  put16(out, e.date);
  put32(out, e.crc);
  put32(out, e.size);
  put32(out, e.size);
  put16(out, (uint16_t)e.name.size());
  put16(out, 0);
  out.write(e.name.data(), (std::streamsize)e.name.size());
  if (!data.empty())
    out.write((const char*)data.data(),
(std::streamsize)data.size());
  if (!out)
    return false;
  entries.push_back(std::move(e));
  return true;
}
static bool createZip(const Config& c)
{
  std::lock_guard<std::mutex> lock(g_archiveMutex);
  fs::path source(c.source), archive(c.archive);
  std::error_code ec;
  if (!fs::is_directory(source, ec))
  {
    logLine(L"Source directory is unavailable: " + c.source);
    return false;
  }
  if (!archive.parent_path().empty())
```

```
    fs::create_directories(archive.parent_path(), ec);
  fs::path temp = archive;
  temp += L".tmp";
  std::ofstream out(temp, std::ios::binary | std::ios::trunc);
  if (!out)
  {
    logLine(L"Cannot create archive: " + temp.wstring());
    return false;
  }
  std::vector<ZipEntry> entries;
  for (fs::recursive_directory_iterator
       it(source, fs::directory_options::skip_permission_denied,
ec),
     end;
     it != end; it.increment(ec))
  {
    if (ec)
    {
      ec.clear();
      continue;
    }
    if (!it->is_regular_file(ec))
      continue;
    fs::path file = it->path();
    bool selected = false;
    std::wstring name = file.filename().wstring();
    std::error_code relEc;
    std::wstring rel = fs::relative(file, source,
relEc).wstring();
    std::replace(rel.begin(), rel.end(), L'\\', L'/');
    for (const auto& pat : c.patterns)
    {
      if (wildcardMatch(pat.c_str(), name.c_str()) ||
        wildcardMatch(pat.c_str(), rel.c_str()))
      {
        selected = true;
        break;
      }
    }
    if (!selected)
```

```
    continue;
  if (lower(fs::absolute(file, ec).wstring()) ==
    lower(fs::absolute(archive, ec).wstring()))
  {
    ec.clear();
    continue;
  }
  if (!addFileToZip(out, file, source, entries))
  {
    out.close();
    fs::remove(temp, ec);
    logLine(L"Failed while writing ZIP.");
    return false;
  }
}
// End of central directory records.
uint32_t centralOffset = (uint32_t)out.tellp();
for (const auto& e : entries)
{
  put32(out, 0x02014b50);
  put16(out, 20);
  put16(out, 20);
  put16(out, 0x0800);
  put16(out, 0);
  put16(out, e.time);
  put16(out, e.date);
  put32(out, e.crc);
  put32(out, e.size);
  put32(out, e.size);
  put16(out, (uint16_t)e.name.size());
  put16(out, 0);
  put16(out, 0);
  put16(out, 0);
  put16(out, 0);
  put32(out, 0);
  put32(out, e.offset);
  out.write(e.name.data(), (std::streamsize)e.name.size());
}
uint32_t centralSize = (uint32_t)out.tellp() - centralOffset;
put32(out, 0x06054b50);
```

```
  put16(out, 0);
  put16(out, 0);
  put16(out, (uint16_t)entries.size());
  put16(out, (uint16_t)entries.size());
  put32(out, centralSize);
  put32(out, centralOffset);
  put16(out, 0);
  out.close();
  if (!out)
  {
    fs::remove(temp, ec);
    logLine(L"Failed to finish ZIP.");
    return false;
  }
  fs::remove(archive, ec);
  ec.clear();
  fs::rename(temp, archive, ec);
  if (ec)
  {
    logLine(L"Cannot replace archive.");
    fs::remove(temp, ec);
    return false;
  }
  logLine(L"Archive refreshed; files included: " +
      std::to_wstring(entries.size()));
  return true;
}
static DWORD WINAPI watchWorker(LPVOID param)
{
  Config* c = (Config*)param;
  HANDLE hDir =
    CreateFileW(c->source.c_str(), FILE_LIST_DIRECTORY,
          FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
          nullptr, OPEN_EXISTING,
          FILE_FLAG_BACKUP_SEMANTICS | FILE_FLAG_OVERLAPPED,
nullptr);
  if (hDir == INVALID_HANDLE_VALUE)
  {
    logLine(L"Cannot watch source directory (check path and
service "
```

```
      L"account permissions): " +
      c->source);
  return 1;
}
HANDLE event = CreateEventW(nullptr, TRUE, FALSE, nullptr);
if (!event)
{
  CloseHandle(hDir);
  return 1;
}
createZip(*c); // initial backup
std::vector<BYTE> buffer(64 * 1024);
while (WaitForSingleObject(g_stopEvent, 0) != WAIT_OBJECT_0)
{
  OVERLAPPED ov{};
  ov.hEvent = event;
  ResetEvent(event);
  DWORD bytes = 0;
  BOOL ok = ReadDirectoryChangesW(
    hDir, buffer.data(), (DWORD)buffer.size(), TRUE,
    FILE_NOTIFY_CHANGE_FILE_NAME | FILE_NOTIFY_CHANGE_DIR_NAME |
      FILE_NOTIFY_CHANGE_SIZE | FILE_NOTIFY_CHANGE_LAST_WRITE |
      FILE_NOTIFY_CHANGE_CREATION,
    nullptr, &ov, nullptr);
  if (!ok && GetLastError() != ERROR_IO_PENDING)
  {
    logLine(L"ReadDirectoryChangesW failed; retrying.");
    Sleep(1000);
    continue;
  }
  HANDLE waits[2] = {g_stopEvent, event};
  DWORD w = WaitForMultipleObjects(2, waits, FALSE, INFINITE);
  if (w == WAIT_OBJECT_0)
  {
    CancelIoEx(hDir, &ov);
    break;
  }
  if (w == WAIT_OBJECT_0 + 1)
  {
    DWORD transferred = 0;
```

```
      if (GetOverlappedResult(hDir, &ov, &transferred, FALSE) &&
        transferred > 0)
      {
        // Debounce a burst of file changes.
        if (WaitForSingleObject(g_stopEvent, c->debounceMs) ==
          WAIT_TIMEOUT)
          createZip(*c);
      }
    }
  }
  CancelIoEx(hDir, nullptr);
  CloseHandle(event);
  CloseHandle(hDir);
  return 0;
}
static bool runWorker()
{
  Config c;
  if (!parseConfig(g_configPath, c))
    return false;
  g_stopEvent = CreateEventW(nullptr, TRUE, FALSE, nullptr);
  if (!g_stopEvent)
    return false;
  HANDLE thread = CreateThread(nullptr, 0, watchWorker, &c, 0,
nullptr);
  if (!thread)
  {
    CloseHandle(g_stopEvent);
    g_stopEvent = nullptr;
    return false;
  }
  // Periodic rescans ensure recovery from missed notifications.
  while (WaitForSingleObject(g_stopEvent, 1000) != WAIT_OBJECT_0)
  {
    static DWORD ticks = 0;
    ticks += 1000;
    if (ticks >= c.rescanSeconds * 1000)
    {
      createZip(c);
      ticks = 0;
```

```
    }
  }
  CancelSynchronousIo(thread);
  SetEvent(g_stopEvent);
  WaitForSingleObject(thread, 5000);
  CloseHandle(thread);
  CloseHandle(g_stopEvent);
  g_stopEvent = nullptr;
  return true;
}
static void setStatus(DWORD state, DWORD win32Exit = NO_ERROR,
            DWORD waitHint = 0)
{
  g_status.dwServiceType = SERVICE_WIN32_OWN_PROCESS;
  g_status.dwCurrentState = state;
  g_status.dwWin32ExitCode = win32Exit;
  g_status.dwServiceSpecificExitCode = 0;
  g_status.dwWaitHint = waitHint;
  g_status.dwControlsAccepted =
    (state == SERVICE_RUNNING)
      ? (SERVICE_ACCEPT_STOP | SERVICE_ACCEPT_SHUTDOWN)
      : 0;
  g_status.dwCheckPoint =
    (state == SERVICE_START_PENDING || state ==
SERVICE_STOP_PENDING) ? 1
                                        : 0;
  if (g_statusHandle)
    SetServiceStatus(g_statusHandle, &g_status);
}
static DWORD WINAPI controlHandlerEx(DWORD control, DWORD, LPVOID,
LPVOID)
{
  switch (control)
  {
  case SERVICE_CONTROL_STOP:
  case SERVICE_CONTROL_SHUTDOWN:
    setStatus(SERVICE_STOP_PENDING, NO_ERROR, 5000);
    if (g_stopEvent)
      SetEvent(g_stopEvent);
    return NO_ERROR;
```

```
  case SERVICE_CONTROL_INTERROGATE:
    if (g_statusHandle)
      SetServiceStatus(g_statusHandle, &g_status);
    return NO_ERROR;
  default:
    return ERROR_CALL_NOT_IMPLEMENTED;
  }
}
static void WINAPI serviceMain(DWORD, LPWSTR*)
{
  g_statusHandle =
    RegisterServiceCtrlHandlerExW(SERVICE_NAME, controlHandlerEx,
nullptr);
  if (!g_statusHandle)
    return;
  setStatus(SERVICE_START_PENDING, NO_ERROR, 5000);
  logLine(L"Service starting.");
  Config initialConfig;
  if (!parseConfig(g_configPath, initialConfig) ||
    !fs::is_directory(fs::path(initialConfig.source)))
  {
    logLine(L"Startup validation failed; check backup.ini and
source_dir.");
    setStatus(SERVICE_STOPPED, ERROR_INVALID_DATA);
    return;
  }
  setStatus(SERVICE_RUNNING);
  bool ok = runWorker();
  logLine(ok ? L"Service stopped." : L"Service failed.");
  setStatus(SERVICE_STOPPED, ok ? NO_ERROR :
ERROR_SERVICE_SPECIFIC_ERROR);
}
static bool manageService(const std::wstring& action)
{
  SC_HANDLE scm = OpenSCManagerW(
    nullptr, nullptr, SC_MANAGER_CONNECT |
SC_MANAGER_CREATE_SERVICE);
  if (!scm)
  {
    std::wcerr << L"OpenSCManager failed: " << GetLastError() <<
```

```
L"\n";
    return false;
  }
  SC_HANDLE svc = OpenServiceW(scm, SERVICE_NAME,
                 SERVICE_QUERY_STATUS | SERVICE_START |
                   SERVICE_STOP | DELETE);
  if (action == L"install")
  {
    if (svc)
    {
      CloseServiceHandle(svc);
      CloseServiceHandle(scm);
      std::wcerr << L"Service already exists.\n";
      return false;
    }
    std::wstring exe = L"\"" + std::wstring(GetCommandLineW()) +
               L"\""; // replaced below with actual executable
path
    std::wstring currentExe(32768, L'\0');
    DWORD exeLen = GetModuleFileNameW(nullptr, currentExe.data(),
                     (DWORD)currentExe.size());
    currentExe.resize(exeLen);
    exe = L"\"" + currentExe + L"\" --service";
    svc = CreateServiceW(scm, SERVICE_NAME, SERVICE_DISPLAY,
               SERVICE_ALL_ACCESS, SERVICE_WIN32_OWN_PROCESS,
               SERVICE_DEMAND_START, SERVICE_ERROR_NORMAL,
               exe.c_str(), nullptr, nullptr, nullptr, nullptr,
               nullptr);
    if (!svc)
    {
      std::wcerr << L"CreateService failed: " << GetLastError() <<
L"\n";
      CloseServiceHandle(scm);
      return false;
    }
    CloseServiceHandle(svc);
    CloseServiceHandle(scm);
    std::wcout << L"Service installed.\n";
    return true;
  }
```

```
  if (!svc)
  {
    std::wcerr << L"OpenService failed: " << GetLastError() <<
L"\n";
    CloseServiceHandle(scm);
    return false;
  }
  bool ok = true;
  if (action == L"remove")
  {
    SERVICE_STATUS ss{};
    ControlService(svc, SERVICE_CONTROL_STOP, &ss);
    if (!DeleteService(svc))
    {
      std::wcerr << L"DeleteService failed: " << GetLastError() <<
L"\n";
      ok = false;
    }
    else
      std::wcout << L"Service marked for deletion.\n";
  }
  else if (action == L"start")
  {
    if (!StartServiceW(svc, 0, nullptr))
    {
      DWORD e = GetLastError();
      if (e != ERROR_SERVICE_ALREADY_RUNNING)
      {
        std::wcerr << L"StartService failed: " << e << L"\n";
        ok = false;
      }
    }
    if (ok)
      std::wcout << L"Start requested.\n";
  }
  else if (action == L"stop")
  {
    SERVICE_STATUS ss{};
    if (!ControlService(svc, SERVICE_CONTROL_STOP, &ss))
    {
```

```
      DWORD e = GetLastError();
      if (e != ERROR_SERVICE_NOT_ACTIVE)
      {
        std::wcerr << L"Stop failed: " << e << L"\n";
        ok = false;
      }
    }
    if (ok)
      std::wcout << L"Stop requested.\n";
  }
  else if (action == L"restart")
  {
    SERVICE_STATUS ss{};
    ControlService(svc, SERVICE_CONTROL_STOP, &ss);
    for (int i = 0; i < 30; ++i)
    {
      SERVICE_STATUS_PROCESS ssp{};
      DWORD needed = 0;
      if (QueryServiceStatusEx(svc, SC_STATUS_PROCESS_INFO,
(LPBYTE)&ssp,
                   sizeof(ssp), &needed) &&
        ssp.dwCurrentState == SERVICE_STOPPED)
        break;
      Sleep(500);
    }
    if (!StartServiceW(svc, 0, nullptr))
    {
      std::wcerr << L"Restart failed: " << GetLastError() <<
L"\n";
      ok = false;
    }
    else
      std::wcout << L"Restart requested.\n";
  }
  else if (action == L"status")
  {
    SERVICE_STATUS_PROCESS ssp{};
    DWORD needed = 0;
    if (QueryServiceStatusEx(svc, SC_STATUS_PROCESS_INFO,
(LPBYTE)&ssp,
```

```
                sizeof(ssp), &needed))
      std::wcout << L"State code: " << ssp.dwCurrentState <<
L"\n";
    else
      ok = false;
  }
  CloseServiceHandle(svc);
  CloseServiceHandle(scm);
  return ok;
}
int wmain(int argc, wchar_t** argv)
{
  g_configPath = joinPath(exeDir(), L"backup.ini");
  g_logPath = joinPath(exeDir(), L"WinBackupService.log");
  std::wstring action = argc > 1 ? lower(argv[1]) : L"";
  if (action == L"install" || action == L"remove" || action ==
L"start" ||
    action == L"stop" || action == L"restart" || action ==
L"status")
    return manageService(action) ? 0 : 1;
  if (action == L"debug")
  {
    std::wcout << L"Debug worker mode. Stop with Ctrl+C.\n";
    SetConsoleCtrlHandler(
      [](DWORD t) -> BOOL
      {
        if (t == CTRL_C_EVENT || t == CTRL_BREAK_EVENT ||
          t == CTRL_CLOSE_EVENT)
        {
          if (g_stopEvent)
            SetEvent(g_stopEvent);
          return TRUE;
        }
        return FALSE;
      },
      TRUE);
    // runWorker creates its own stop event; Ctrl+C is primarily
useful for
    // manual termination.
    return runWorker() ? 0 : 1;
```

```
  }
  SERVICE_TABLE_ENTRYW table[] = {
    {const_cast<LPWSTR>(SERVICE_NAME), serviceMain}, {nullptr,
nullptr}};
  if (!StartServiceCtrlDispatcherW(table))
  {
    DWORD e = GetLastError();
    std::wcerr << L"Not started by SCM (error " << e
           << L"). Use install/start or debug.\n";
    return 1;
  }
  return 0;
}
```